Contextualized - Project Shift to MCP

Overview

The goal of this document is to go through and outline the updates that will be made to the `Contextualized` project in order to account for the new Agentic logic that we want to include while querying for information.

High Level Changes

- Shift from a RAG centric application to an Agentic approach that has RAG capabilities
- Usage of MCP Server that will store relevant tools that the LLM can leverage when attempting to answer the suers question

Components

- **Frontend**
  - Renders chat
  - Handles **Streaming Events**
    - Informs end user about “Calling Tool X”, or “Searching”
  - UI to configure project, data sources, and manually run ingestion jobs
  - Ability to “connect” existing MCP servers (internal OR external)
- **Backend**
  - The “Orchestrator”
    - Endpoints for adding / editing / deleting our relevant database models
    - Defines internal tools that we be leveraged by agent to answer question
      - **Do these belong in our own “internal tools” MCP?**
    - Acts as **MCP Client**
- **MCP Layer**
  - A collection of external “Capabilities”
  - These are linked via our linking functionality, allows for users to connect to curated list

- Handles security and safety
- When a user adds “GitHub” data source, an associated link to the MCP server should be made

MCP Integrations

When configuring a `DataSource` for a particular Project, an associated MCP Server “can/should” be also configurable. This shouldn’t be “enforced” (as there could examples down the line where we want to simply ingest “Plain Files”, but it should be allowed

Internal vs External MCPs

LLamaIndex will be able to serve as the bridge between our MCP servers and our Agentic Loop.

- **External Tool**
  - ```
    mcp_tool_spec = BasicMCPClient(url="https://api.internal-tools.com/mcp")
    ```
- **Internal Tool**
  - ```
    mcp_tool_spec = BasicMCPClient(
        command="npx",
        args=["-y", "@modelcontextprotocol/server-github"]
    )
    ```
- We can then leverage the **tools** available by these MCP and pass these to the `agentic loop`
  - `mcp_tool_spec.to_tool_list()`
  - The output of this above command can be leveraged when we pass the list of available tools to our `FunctionCallingAgentWorker`

Configuring MCP’s

- We will need to expand on our “DataSource” logic to configure or link to an existing MCP server
- We can create a **mcp_config** table that a `DataSource` can link to upon creation
- The

Backend

Overview

The goal here is to shift away from the current pattern of

- **Prompt → Decomposition of Query → Retrieve Chunks for Each Sub Query (BM25 & Chroma) → Provide Conversation History & Chunks to LLM → Generate Response**

To the following

- **Prompt → Get Tools Based on The Configured Project → Configure Agent → Pass Prompt to Agent (Conversation History too?) → Agentic Cycle (Calls to MCP Server, Utilization of Internal Tooling) → Stream of Updates Back to End User → Stream Final Response to End User**

Agentic Cycle

- **Steps**
  - 1) Retrieve the `DataSource`'s associated with the Project
  - 2) Retrieve the associated `McpConfigs` associated with those `DataSource`'s
  - 3) Initialize MCP sessions
  - 4) Call `mcp_client.list_tools()` and ensure that config isn't stale and server is avialable
  - 5) Inject the mcp tools, and corresponding internal tools, into our agent
  - 6) Pass instruction set outlined in our `retrieval.md` file
- **Context Management**
  - Too many tools available across the three distinct Data Sources configured for the project
  - **Solution:** Only inject the tool definitions based on the User's intent (**Dynamic Tool Loading,** not just adding them all each time)
  - Conversation History
    - Conversation history grows large over time
    - We should leverage the **Janitor** that will summarize Conversation History in smaller number of words in order to avoid the over usage of the context window
- **Llama Index Integration**
  - Depending on the **Provider** selected, the corresponding LlamaIndex wrapper will need to be leveraged
    - **OpenAI / Claude / Gemini / Etc**
      - Latest models available for these support "native tool calling" that they were trained to do during training

- Due to this, they can leverage the `FunctionCallingAgentWorker`
  - ```
    worker = FunctionCallingAgentWorker.from_tools(
        [status_tool],
        llm=llm,
        verbose=True
    )

    agent = AgentRunner(worker)

    response = agent.chat("What is the status of the Direct Deposit project?")
    ```

- **Ollama**
  - Need to leverage `ReActAgent`, which will effectively provide "agentic reasoning" for LLMs that were not trained to do this via a *text-based loop* where the LLM talks to itself
  - ```
    agent = ReActAgent.from_tools(
        [status_tool],
        llm=llm,
        verbose=True  # see "Thought Process" as output
    )

    response = agent.chat("What is the status of the Direct Deposit project?")
    ```

- **Streaming Events**
  - For good periodic updates based on response from agent, we'll likely need to use `AgentWorkflow`
  - The `AgentWorkflow` can be wrapped around both the `ReActAgent` and the `FunctionAgent`
    - ```
      return AgentWorkflow(agents=[agent], root_agent=agent_config.name)

      handler = workflow.run(user_msg=user_msg)

          async for event in handler.stream_events():
              # This works for BOTH ReAct and FunctionCalling agents!
              if isinstance(event, ToolCall):
                  yield f"data: {{'status': 'Checking {event.tool_name}...'}}\n\n"

              elif isinstance(event, AgentStream):
                  yield f"data: {{'token': '{event.delta}'}}\n\n"
      ```

Frontend

- **Project Configuration**
  - Allow for additional entries such as:
    - Jira Epics
    - ?
- **MCP Configuration**
  - Need some tab for available MCP servers
      ```

- Ability to associate MCP to Data Source
- There should be a “all-in-one” flow (add data source, prompted to add corresponding MCP or select existing one), or the ability to go back and add only a data source, and later down line add new MCP

Agents

1. `retrieval.md`
   a. Core instruction set about agents responsibility and its need to use existing Conversation history and tools to answer users question
2. `unit_testing.md`
   a. This will be a agent to create unit tests on the functionality that we are adding
3. `ui.md`
   a. The instruction set for creating or updating new UI components